

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Networking and SSH Administration

Unit 10 • Linux (10-151-105)



Learning Outcomes

- **Execute shell commands** to view and configure network interfaces, routing, and addressing.
- **Use documentation** (man pages, --help) to confirm command syntax for unfamiliar networking tools.
- **Use a text editor** to edit configuration files such as the Netplan YAML and `sshd_config`.
- **Modify configuration files** to set a static address, define name resolution, and harden the SSH service.
- **Apply secure remote administration** with key-based authentication between hosts.



Competency 2

Command-line administration — the unit anchors C2.

LESSON 1

Networking Fundamentals



How a Host Is Identified

- **MAC address** — a permanent hardware ID burned into the network card; switches use it within a segment.
- **IP address** — a logical address like `192.168.10.20`, paired with a subnet mask (`/24`) that splits network from host.
- **Hostname** — a human-friendly name like `rocky10` that's easier to remember than digits.

Reads it for you

`ip addr show` (or `ip a`)
lists address, mask, and MAC
for every interface.

Ports name the service

22 SSH · 53 DNS · 80 HTTP
443 HTTPS · 25 SMTP

Static vs. Dynamic Addressing

	Static	Dynamic (DHCP)
Set by	Administrator, by hand	Leased automatically from a server
Address	Fixed — never changes	May change over time
Best for	Servers, routers, printers	Laptops, desktops, phones
In your pod	What you configure this week	Concept only — pod is isolated

Servers need a stable identity; clients that come and go do not.

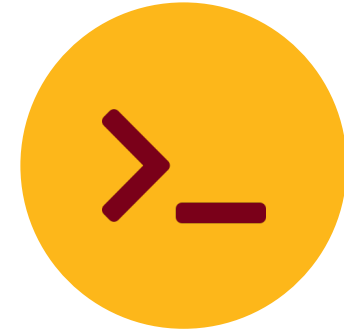
LESSON 2

Configuring the Network



Viewing the Network: the ip Command

- **ip addr show** — addresses, masks, and MAC for every interface (abbreviate as `ip a`).
- **ip link** — interface state; toggle with `ip link set <name> up/down`.
- **ip route show** — the routing table, including the default gateway.



Legacy tools

ifconfig, route, and netstat still appear in old docs but are deprecated — learn ip and ss.

Rocky: NetworkManager and nmcli

- **nmcli connection show** — list saved connection profiles.
- **nmcli device status** — hardware interfaces and their state.
- **nmcli connection up / down <name>** — activate or deactivate a connection.
- **Set a static address:** assign address + prefix, gateway, DNS; switch the profile to manual; reactivate.



Prefer menus?

nmtui gives the same settings in a keyboard-driven screen.

Ubuntu: Netplan

- **Edit the YAML:** set `dhcp4: no`, the static `addresses` (CIDR), the route, and `nameservers`.
- **`sudo netplan try`** — applies temporarily; auto-reverts after 120 s unless you confirm.
- **`sudo netplan apply`** — makes it permanent once you know it works.



YAML caution

Indentation is structural.
Use spaces, never tabs
(two per level). One stray
space breaks the file.

Two Ecosystems, One Goal

	Rocky Linux 10	Ubuntu Server 24.04
Tool	NetworkManager	Netplan
Configure via	nmcli (or nmtui menu)	A YAML file in /etc/netplan
Source of truth	The connection profile	The .yaml file
Apply	nmcli connection up <name>	netplan try, then netplan apply
Safety net	Re-activate the prior profile	netplan try auto-reverts (120 s)
Verify	ip a / nmcli connection show	ip a / netplan status

LESSON 3

Name Resolution & Diagnostics



Name Resolution: Names to Addresses

- **/etc/hosts** — local name-to-address map, checked first, no server needed.
- **/etc/resolv.conf** — the DNS servers to query when a name isn't local.
- **/etc/nsswitch.conf** — the lookup order: hosts first, then DNS.



Classic test

Ping by name, then by IP.
If IP works but the name
doesn't, networking is fine —
the problem is resolution.

The Simulated Lab Internet

What it lets you do

Point a resolver at a real DNS server and watch `dig` / `nslookup` return answers.

Run `ping google.com` and `ping 8.8.8.8` and get replies — once the DNS client is set.

Keep the truth in mind

Those replies come from inside your pod, not from Google.

It is a faithful simulation for practice — not a live link to the outside world. The lesson is the workflow, not the destination.

DNS Query Tools

- **dig <name>** — most detail: the answer, which server replied, and timing.
- **host <name>** and **nslookup <name>** — quick, simple lookups.
- **resolvectl query <name>** — the systemd tool for the same job.

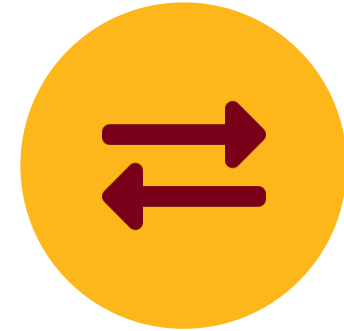


Against the lab DNS

Point the resolver at the pod DNS server, then query it.

Confirming Connectivity

- **ping -c 4 <dest>** — reachability. **-c** limits the count; **Ctrl+C** stops a running ping.
- **traceroute / tracepath <dest>** — the path hop by hop (tracepath needs no privileges).
- **mtr <dest>** — live ping + traceroute with per-hop packet loss.



Targets in the pod

The other VM, the gateway, and the simulated internet all reply.

Seeing What's Listening: ss

- **-t** TCP · **-u** UDP · **-l** listening only · **-n** numeric · **-p** owning process.
- **ss -tlnp** — the combination you'll use constantly: every listening TCP port with its process.
- **Read the result:** a missing socket means the service isn't running; an unexpected one means something is listening that maybe shouldn't be.



Later this week

Use ss to confirm sshd is listening on port 22.

LESSON 4

Remote Administration with SSH



SSH: Encrypted Remote Access

- **ssh student@ubuntu24** — open a session as a user on a host; add **-p** <port> for a non-default port.
- **First connection:** confirm the server's fingerprint; it's saved to `~/.ssh/known_hosts` so SSH can detect impersonation later.



Pod-to-pod

Connect Rocky ↔ Ubuntu inside the pod — no internet needed. SSH is ideal on a closed net.

A Dual-Distro Divergence



	Rocky Linux 10	Ubuntu Server 24.04
Service unit	sshd.service	ssh.service
Manage with	systemctl status sshd	systemctl status ssh
In your pod	OpenSSH already installed	Install it: sudo apt install openssh-server

Then enable and start the service so it survives a reboot.

Key-Based Authentication

- 1. **ssh-keygen** — generate the pair (private + `.pub`) in `~/.ssh`.
- 2. **ssh-copy-id student@ubuntu24** — append your public key to the server's `authorized_keys`.
- 3. **ssh student@ubuntu24** — connect; no password prompt this time.

Two key files

`authorized_keys`
— on the SERVER; public keys
allowed to log in.

`known_hosts`
— on the CLIENT; public keys of
servers you've connected to.

Hardening the SSH Server

- **PermitRootLogin no** — no direct root login; connect as a user and use `sudo`.
- **PasswordAuthentication no** + **PubkeyAuthentication yes** — require keys (only after keys work).
- **AllowUsers** / **AllowGroups** — limit SSH to named accounts; list your own first.
- **X11Forwarding no** — don't tunnel a GUI a server doesn't need.



Apply

```
systemctl restart sshd (Rocky)
systemctl restart ssh (Ubuntu)
```

Security Focus: Lock the Front Door

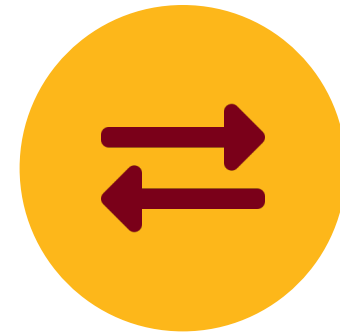


- **Prefer keys over passwords** — a private key can't be guessed and isn't transmitted.
- **Disable root login over SSH** so the one universal account can't be targeted.
- **Turn off password auth** once keys work — this ends password-guessing attacks.
- **Audit listening ports with** `ss -tlnp` — you can't secure an exposure you can't see.

Why it matters: a firewall decides what traffic reaches your services; this week makes sure the services themselves are locked down.

Transferring Files over SSH

- **scp** `<src> user@host:/path` — copy a file over SSH; the quick choice for a one-off.
- **sftp** `user@host` — an interactive session to list, upload, download, and manage files.
- **rsync** — copies only the differences; great for repeated syncs and backups (met briefly in Unit 5).



Pod-to-pod

Move files between Rocky and Ubuntu without leaving the pod.

LESSON 5

Beyond the Command Line



Remote Access Beyond the CLI

- **Cockpit** — a web console at `https://<host>:9090` for services, storage, logs, and networking.
- **VNC** and **XRDP** — full remote desktops (XRDP speaks Microsoft's RDP).
- **X11 forwarding** — tunnel a single remote GUI program back through SSH (`X11Forwarding`).



A LOOK AHEAD

Awareness now, hands-on later —
a map of what's out there.

A First Look at Visibility Tools

- **nmap** — scans for live hosts, open/closed/filtered ports, and OS guesses. The administrator's audit tool — and the attacker's first move.
- **tcpdump** / **Wireshark** — capture and read the actual traffic on an interface.
- **netcat (nc)** — open and test raw connections; the 'Swiss Army knife' of networking.



Authorization rule

Only scan or capture on networks you own or are authorized to test. Your isolated pod qualifies — a safe place to begin.

Your Hands-On Lab

- Assign and verify a static address on both hosts.
- Configure name resolution — /etc/hosts and the lab DNS server.
- Run the connectivity and socket diagnostics (ping, ss).
- Generate keys and enable key-based login between the machines.
- Harden the SSH service and confirm it still lets you in.



Snapshot

WK10_NETWORKING

Key Takeaways

- **View with** `ip`, configure with `nmcli` (Rocky) or `Netplan` (Ubuntu).
- **Same goal, different dialect** — learn the shared steps, not two command lists.
- **Resolve names** with `/etc/hosts` and DNS; test with ping-by-name-then-by-IP.
- **SSH is the front door** — use keys, disable root and passwords, and never lock yourself out.
- **Know what's listening** with `ss` — the setup for the Unit 11 firewall.

Questions?

Unit 10 · Networking and SSH Administration

